

VRX 环境下 WAM-V 自主航点导航系统设计

平台搭建与基础闭环控制

ROS 2 框架

Gazebo 仿真

航点任务

闭环控制

汇报主线：搭建仿真环境 → 打通控制接口 → 编写自主控制器 → 完成基础任务

本阶段工作一：搭建环境并打通控制接口

第一次组会

本页重点：上方是我完成的三项工程工作，下方是这些工作最终打通的控制闭环。

01 完成仿真环境搭建

编译 ROS 2 Humble 与 VRX 工作区
配置 Gazebo Garden 资源路径
验证 WAM-V 模型、传感器与世界
制作无竞赛倒计时学习场景

02 完成控制接口梳理

定位 GPS、IMU、雷达和航点话题
确认消息类型与服务质量策略
测试推进器方向和推力正负号
明确任务开始、运行与结束状态

03 编写一键启动脚本

自动加载 ROS 2 与工作区环境
等待关键话题真正收到数据
启动仿真后自动运行控制器
退出时安全停止并清理进程

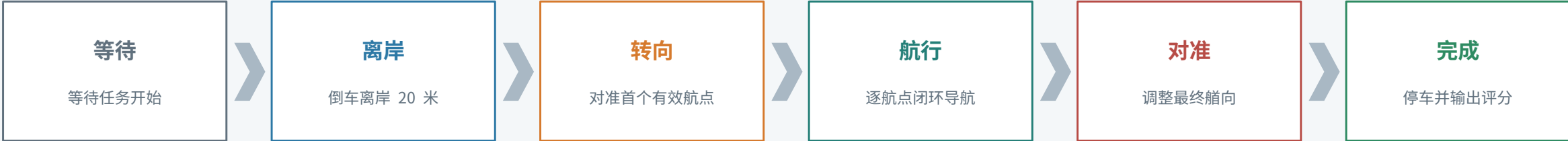
在此基础上打通的闭环链路



本阶段工作二：实现第一版自主控制器

第一次组会

本页重点：上方是我设计的任务状态机，左下是控制算法，右下是实际仿真结果。



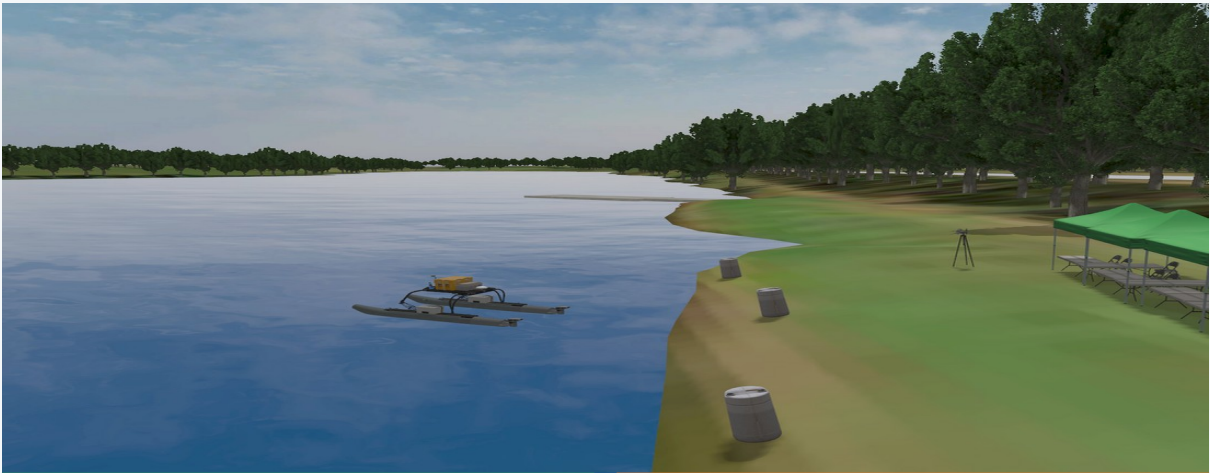
代码证据：autonomous_controller.py (881 行) | launch_wayfinding.sh (70 行)

控制律与安全约束

航向误差 $e\psi = \text{目标航向 } \psi_d - \text{当前航向 } \psi$
控制量 = 比例项 + 积分项 + 微分项

左推力 = 前进量 - 转向量 右推力 = 前进量 + 转向量

- 积分与输出限幅，避免控制量持续饱和
- 传感器未就绪或任务结束时，推力强制归零
- 雷达危险区停止前进，保留差动转向能力



阶段成果

完成自动离岸、航点切换、艏向对准

下一步

提升定位稳定性，建立障碍记忆与可行路径

阶段总结：我做了什么、解决了什么、学到了什么

第一次组会

导师视角：本页把项目已有平台与我新增的工作分开，并给出可验证成果和能力成长。

我完成了什么

代码与工程产出

- 搭建并编译 ROS 2、Gazebo 与 VRX 仿真环境
- 梳理传感器、任务与推进器接口及消息类型
- 编写 881 行自主控制器和一键启动脚本
- 实现坐标转换、反馈控制、雷达避障与视觉检测
- 实现离岸、航行、对准、完成等任务状态机

解决了什么问题

从“能启动”到“能自主完成任务”

- 解决出生点靠岸导致启动碰撞的问题
- 解决推进器极性和左右差动方向不明确的问题
- 解决任务、航点和传感器就绪不同步的问题
- 解决航点捕获附近反复切换和最终艏向对准问题
- 解决退出后残留进程继续发布推力的问题

我学到了什么

理论知识与工程能力

- 掌握 ROS 2 发布订阅、服务质量策略和启动系统
- 理解坐标系、经纬度换算和四元数航向提取
- 掌握反馈控制、抗积分饱和和差动推力分配
- 学会用状态机组织复杂任务与安全条件
- 形成从日志、话题和仿真现象定位问题的方法

验证证据：无需键盘遥控即可完成自动离岸、航点切换与最终艏向对准；下一阶段重点解决定位噪声和环境记忆。